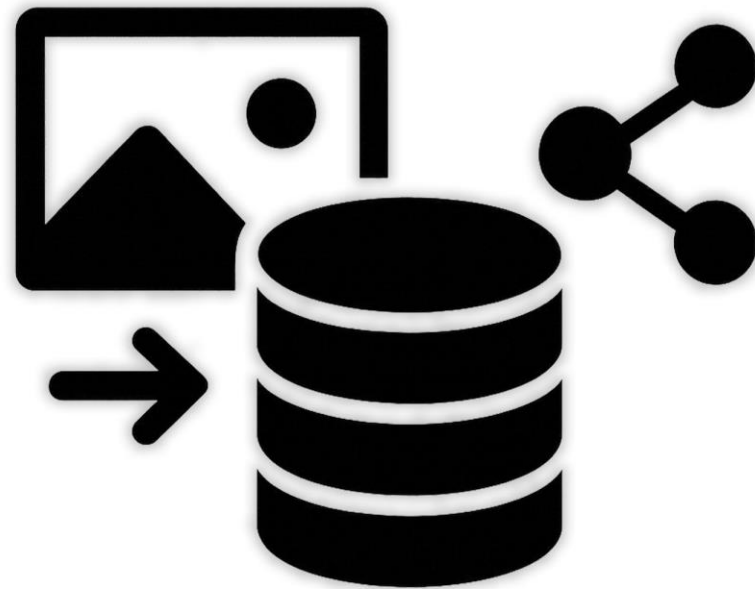
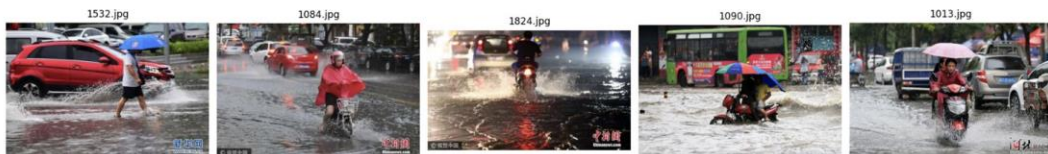




BIGDATA



ZAYD BADLA - 949904
ELIAS MAHJOUBI - 947650

HOCHSCHULE DÜSSELDORF
BIG DATA - SOSE 25

1. Motivation/Aim of the project	2
1.1 Unsere Motivation	2
1.2 Unser Ziel	2
2. Program Design	3
2.1 Hauptbestandteile und Zusammenspiel	3
2.2 Eingesetzte Techniken und Bibliotheken.....	5
3. Image Similarity.....	6
3.1 Farbähnlichkeit mit HSV-Histogrammen	6
3.2 Deep-Learn Embedding.....	8
3.3 Histogram of Oriented Gradients	11
3.3 Ergebnisbewertung der Suchmethoden.....	14
3.4 Zusammenführung	15
4 Leistungsanalyse und Laufzeitoptimierung.....	16
4.1 Engpässe und Messergebnisse	16
4.2 Optimierungen.....	17
5. Feasibility analysis/Discussion.....	18
6. Big Data Image Analysis	20
6.1 Ergebnisse	20
6.1 Diskussion	21

1. Motivation/Aim of the project

1.1 Unsere Motivation

Unsere Motivation für dieses Projekt liegt in der Neugier am Fach Big Data. Wir möchten verstehen, wie Bildähnlichkeit in großen Datenbeständen berechnet werden kann und welche Methoden sich dabei bewähren. Besonders interessiert uns, welche Verfahren am schnellsten und zuverlässigsten arbeiten und an welchen Stellen man an die praktischen Grenzen von Speicher, Rechenzeit und Genauigkeit stößt. Das Projekt bietet uns die Gelegenheit, verschiedene Ansätze direkt miteinander zu vergleichen und dadurch ein tieferes Verständnis für die Stärken und Schwächen der unterschiedlichen Methoden zu entwickeln.

1.2 Unser Ziel

Das Ziel unseres Projekts ist die Entwicklung einer Python Software, die für ein Eingabebild die fünf ähnlichsten Bilder aus einem sehr großen Datensatz finden kann und dabei sowohl Genauigkeit als auch Geschwindigkeit berücksichtigt. Dafür wollen wir drei verschiedene Verfahren einsetzen und untersuchen, wie sie sich in Bezug auf Effizienz, Aussagekraft und Skalierbarkeit unterscheiden. Außerdem möchten wir herausfinden, ob eine Kombination der Methoden möglich ist und welche Form der Suche in der Praxis die besten Ergebnisse liefert. Auf diese Weise verbinden wir theoretisches Wissen mit praktischer Umsetzung und nähern uns der Frage, wie ein Bildrecommender für Big Data Szenarien gestaltet sein sollte.

2. Program Design

2.1 Hauptbestandteile und Zusammenspiel

Das System besteht aus mehreren klar abgegrenzten Komponenten, die über eine gemeinsame Datenbasis miteinander verbunden sind. Den Einstieg bildet das Modul `image_indexer.py`, das rekursiv Bildpfade aus dem Dateisystem lädt und dabei unerwünschte System- oder Trash-Ordner überspringt. Für jedes Bild wird eine stabile 63-bit Kennung auf Basis des Pfads mit `blake2b` erzeugt, sodass die Zuordnung unabhängig von der Reihenfolge des Einlesens eindeutig bleibt. Zusätzlich werden Dateimetadaten wie Größe und Änderungszeitpunkt ausgelesen. Die Einträge werden anschließend in einer SQLite-Datenbank gespeichert oder bei Änderungen aktualisiert. Dabei werden Features wie HSV- oder HOG-Vektoren automatisch zurückgesetzt, sobald sich Metadaten geändert haben, damit die zugehörigen Merkmale bei Bedarf neu berechnet werden. Für eine hohe Performance kommen während der Bulk-Operationen optimierte PRAGMAs zum Einsatz, unter anderem Write-Ahead-Logging, temporäre Speicherung im Speicher und ein erhöhter Cache. Gleichzeitig wird die Threadanzahl in den verwendeten BLAS- und OpenCV-Bibliotheken auf eins begrenzt, um Kollisionen mit dem späteren Multiprocessing zu vermeiden. Über das Command-Line-Interface lässt sich diese Indexierung gezielt steuern, wobei beispielsweise der Stage index für den Scan und die Eintragung der Metadaten zuständig ist, während `purge` das Entfernen unerwünschter Pfade übernimmt.

Aufbauend auf dieser Datenbasis erfolgt die Feature-Extraktion in separaten Skripten. Für HSV- und HOG-Deskriptoren werden die Merkmalsvektoren direkt in der Datenbank gespeichert, indem die Spalten `hsv_vector` und `hog_vector` als BLOB-Felder gefüllt werden. Die tiefergehenden Deep-Learning-Embeddings mit EfficientNet-B0 werden dagegen nicht in der Datenbank gespeichert, sondern als externe `.npy`-Dateien in einem Sharding-Schema abgelegt. Die Verknüpfung zwischen Datenbankeinträgen und Embeddings erfolgt dabei über die Bildkennung, sodass beide Datenquellen konsistent bleiben.

Für die Suche wird je nach Verfahren auf unterschiedliche Strategien zurückgegriffen. HSV- und HOG-Vektoren können direkt in der Datenbank linear verglichen und gerankt werden, was kompakt und schnell umsetzbar ist. Für die Deep-Learning-Embeddings wird hingegen ein separater Index aufgebaut, der über FAISS mit einem

HNSW-Graphen arbeitet und dadurch eine sehr schnelle Approximate Nearest Neighbor-Suche ermöglicht. Der gesamte Datenfluss lässt sich damit in einer Kette beschreiben, die vom Dateisystem über das Indexing in SQLite führt, dort um klassische Merkmale erweitert und durch externe Embeddings ergänzt wird. Aus diesen Informationen werden schließlich Suchindizes erzeugt, die in den Query-Skripten wiederum genutzt werden, um Ergebnisse aus allen Verfahren oder in Kombination bereitzustellen.

2.2 Eingesetzte Techniken und Bibliotheken

Für die Umsetzung wurden mehrere etablierte Bibliotheken eingesetzt. Die gesamte Pipeline ist in Python 3 implementiert. OpenCV übernimmt zentrale Aufgaben beim Bild-Input, der Größenanpassung, der Farbraumkonvertierung sowie der Berechnung der HOG-Deskriptoren. NumPy wird als Grundlage für numerische Operationen und Vektorisierungen verwendet. Für die Berechnung der Embeddings setzen wir PyTorch ein, wobei wir durch Mixed Precision mit FP16 und Automatic Mixed Precision (AMP) die GPU-Auslastung optimieren.

Für die schnelle Annäherungssuche auf Basis der Embeddings nutzen wir FAISS mit einem HNSW-Index, der eine performante Approximate Nearest Neighbor-Suche ermöglicht. Die Metadatenverwaltung erfolgt in SQLite, das durch gezielte Performance-PRAGMAs wie Write-Ahead-Logging, Speicheroptimierung und Batch-Commits beschleunigt wurde. HSV- und HOG-Vektoren werden direkt als BLOB-Felder in dieser Datenbank gespeichert, während die Embeddings als externe .numpy-Shards organisiert sind. Multiprocessing kommt für die parallele Verarbeitung großer Bildmengen zum Einsatz, wobei durch Thread-Capping ein stabiles Zusammenspiel mit den Bibliotheken sichergestellt ist. Ergänzend verwenden wir tqdm zur Fortschrittsanzeige und pytest für automatisierte Tests der Kernfunktionen.

3. Image Similarity

3.1 Farbähnlichkeit mit HSV-Histogrammen

Als erste Methode zur Ähnlichkeitssuche haben wir eine farbbasierte Lösung entwickelt, die die Bilder in den HSV-Farbraum überführt und dort kompakte Histogrammvektoren bildet. Wir normalisieren jedes Bildformat auf eine einheitliche Größe, konvertieren nach HSV und berechnen ein zweidimensionales HS-Histogramm, das wir um ein separates V-Histogramm ergänzen. Damit unterscheiden wir zuverlässig farbarme und Graustufenbilder, ohne sie mit bunten Motiven zu verwechseln. Diese Extraktion ist in [compute_hsv_vector](#) umgesetzt. Wir wenden eine weiche Maske an, die sehr dunkle oder entsättigte Pixel zunächst ausschließt und nur bei Bedarf lockert; fällt trotzdem zu wenig Bildinhalt an, lassen wir die Maske weg. Anschließend normalisieren wir die Histogramme mit der L2-Norm und fügen HS und V zu einem einzigen Merkmalsvektor zusammen.

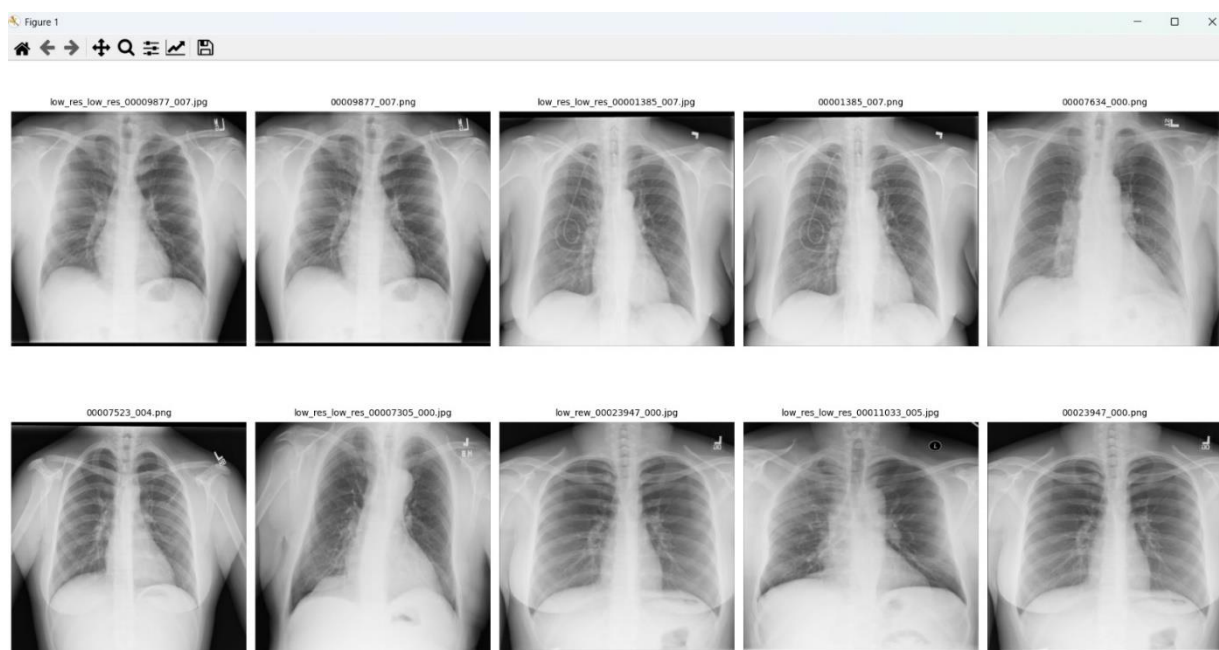
Damit die Vektoren für den gesamten Korpus entstehen, haben wir eine multiprocessing-Pipeline gebaut, die die zu verarbeitenden Bilder seitenweise aus der Datenbank streamt, parallel die Features berechnet und die Ergebnisse paketweise zurückschreibt. Wir initialisieren die Worker prozesssicher, deaktivieren interne Threadpools von OpenCV und vergeben stabile Seeds, damit die Laufzeit gleichmäßig bleibt. Die eigentliche Verarbeitung geschieht in [process_hsv](#), die Workerlogik liegt in [process_feature_worker](#), und die BLOB-Updates in [write_features](#). Für robuste Dateiimporte lesen wir primär mit OpenCV und fallen bei exotischen Formaten auf Pillow zurück, was in [_imread_any](#) steckt.

Für die Suche normalisieren wir Query- und Korpusvektoren auf Einheitslänge und verwenden das Skalarprodukt, was der Kosinusähnlichkeit entspricht. Wir haben dafür eine kleine Indexklasse HSVIndex, die entweder eine RAM-Matrix nutzt oder auf einen FAISS-Index zugreift. Die exakte RAM-Variante erledigt die Suche mit einer Matrixmultiplikation, während die FAISS-Variante optional als IVF-Index arbeitet und damit auf sehr großen Datenbeständen deutlich schneller ist. Das Laden und Vorbereiten des Indexes geschieht in [load_hsv_index](#). Hier prüfen wir zunächst, wie viele Vektoren bereits vorliegen, rekonstruieren sie aus den gespeicherten BLOBs mit [_vec_from_blob](#) und gleichen ältere Vektordimensionen ab, indem wir das V-Histogramm bei Bedarf anhängen. Wenn FAISS aktiv ist, trainieren wir bei großen Datenmengen einen IVF-Index, fügen die Vektoren hinzu und speichern den Index optional auf die Platte; die Fingerprinting-Logik für stabile Dateinamen findet sich in [_index_fingerprint](#), [_faiss_path](#), [save_faiss_index](#) und [try_load_faiss_index](#).

Für die eigentliche Abfrage berechnen wir zunächst den Query-Vektor identisch zur Batchverarbeitung, normalisieren ihn und rufen dann `HSVIndex.search` auf. Dies ist in `find_similar` gekapselt, sodass wir mit einem Pfad und einer gewünschten Trefferzahl sofort Ergebnisse erhalten. Die Bedienung läuft wie bei den anderen Methoden über das Command Line Interface. Mit `--stage hsv` starten wir die parallele Feature-Extraktion und das Speichern in der Datenbank, und mit `--stage search` führen wir eine Suche gegen den RAM- oder den FAISS-Index aus. Die Schalter `--faiss_ivf` und `--persist_index` steuern, ob wir den beschleunigten IVF-Pfad nutzen und ob der Index auf die Platte geschrieben wird, was wir in `load_hsv_index` und den FAISS-Hilfsfunktionen genau so umgesetzt haben.

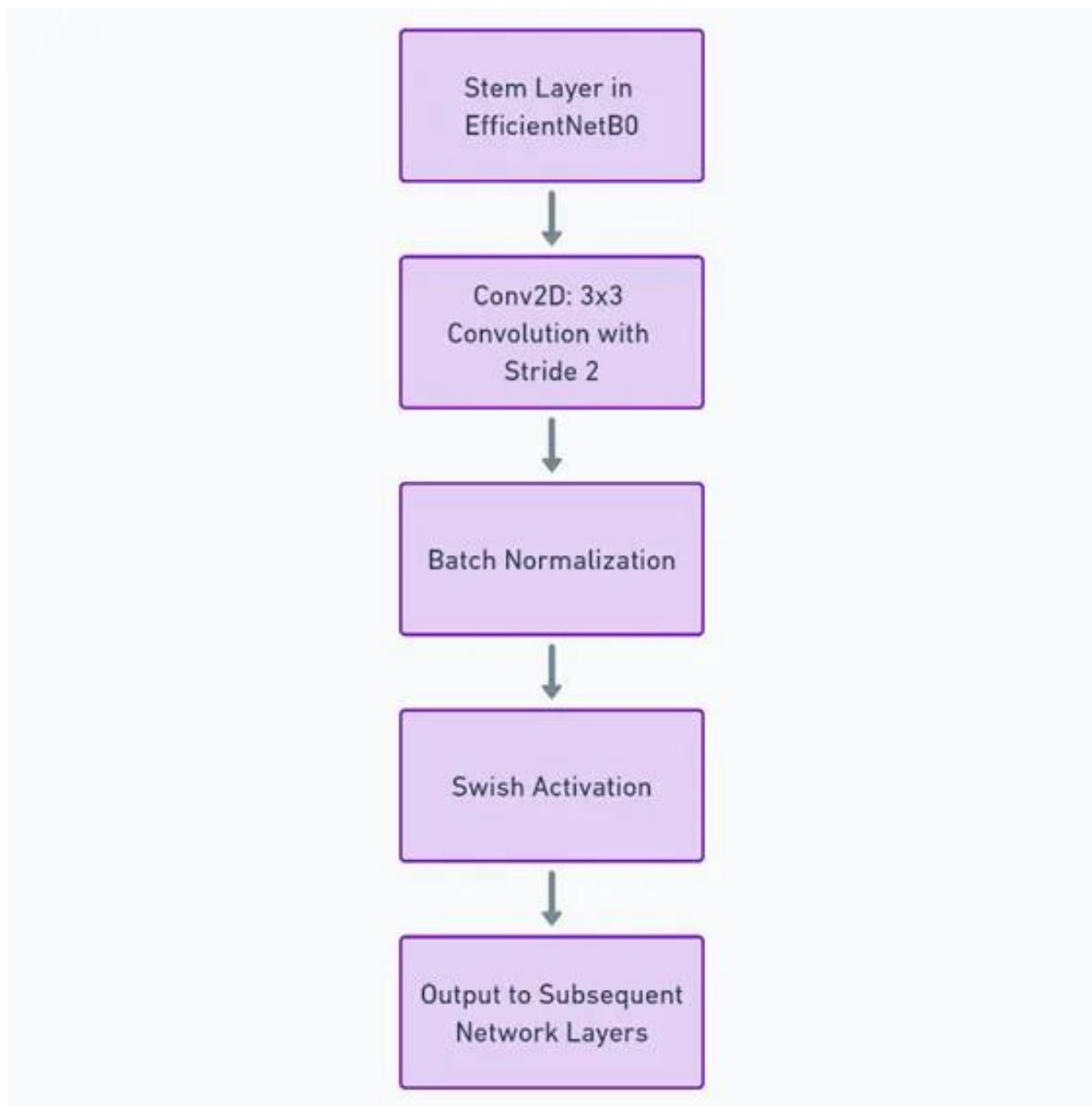
Ergebniss:

Das Bild „low_res_low_res_00009877_007.jpg“ oben links ist das Ausgangsbild.



3.2 Deep-Learn Embedding

Wir benutzen für die Bildähnlichkeitsmessung das Modell EfficientNet-B0, das durch Compound Scaling eine hohe Genauigkeit bei geringer Rechenlast erreicht. Es verarbeitet Bilder zunächst in einer Stem-Schicht mit Convolution, Normalisierung und Swish-Aktivierung, bevor in tieferen Schichten komplexere Strukturen extrahiert werden. Am Ende entsteht ein 1280-dimensionalen Vektor, der das Bild kompakt repräsentiert und als Grundlage für die Ähnlichkeitssuche dient.



Für die skalierbare Ähnlichkeitssuche über hunderte Millionen Dateien haben wir unsere EfficientNet-B0-Embeddings aus dem RAM in ein dateibasiertes Format ausgelagert und mit einem Sharding-Schema organisiert. Wir laden das Backbone einmalig über `_load_backbone`, entfernen den Klassifikationskopf und schalten auf Evaluationsmodus mit optionaler Mixed Precision auf der GPU. Für jedes Bild bereiten wir die Daten mit `_preprocess` vor, indem wir auf 224×224 skalieren, nach ImageNet-Statistik normalisieren und einen Tensor erzeugen. Die eigentliche Feature-Extraktion bündeln wir in `_infer_batch`, wo wir Batches wahlweise auf die GPU verschieben, die Vorwärtsrechnung durchführen, die Ausgaben nach L2 normieren und sie als float32 zurückgeben.

Damit diese Vektoren platzsparend und robust abgelegt werden, schreiben wir sie nicht in die Datenbank, sondern als .npy-Dateien auf die Festplatte. Wir haben dafür ein Sharding konzipiert, das die Dateien nach dem Rest der `image_id` in Unterordner verteilt. Die Konstruktion der Zielpfade erledigen `_feature_dir` und `_numpy_path`, die Speicherung übernimmt `_save_npy_atomic`, sodass wir auch bei Abbrüchen konsistente Dateien behalten. Ob ein Embedding bereits existiert, prüfen wir mit `_exists_embed`, was uns erlaubt, die Pipeline jederzeit wiederaufzunehmen, ohne Arbeit zu duplizieren.

Die Berechnung selbst führen wir mit einer Producer-Consumer-Pipeline aus. Wir sammeln zunächst alle fehlenden `image_id`-Pfad-Paare und geben die Gesamtzahl für eine saubere Fortschrittsanzeige aus. Dann starten wir den Producer-Thread, der mit einem `ThreadPoolExecutor` parallel die Bilder lädt und vorverarbeitet und die Ergebnisse in eine Queue legt. Im Consumer laufen wir über die Queue, bauen dynamische Mini-Batches auf und rufen `_infer_batch` auf. Wir speichern die Ergebnisse unmittelbar mit `_save_npy_atomic` weg und passen die Batchgröße an, wenn die GPU ein OOM meldet. Diese gesamte Logik liegt in `process_embeddings`, wo wir auch die PRAGMAS für SQLite setzen, den Ausgabepfad bestimmen und das Speicherdatenformat zwischen fp16 und fp32 wählen.

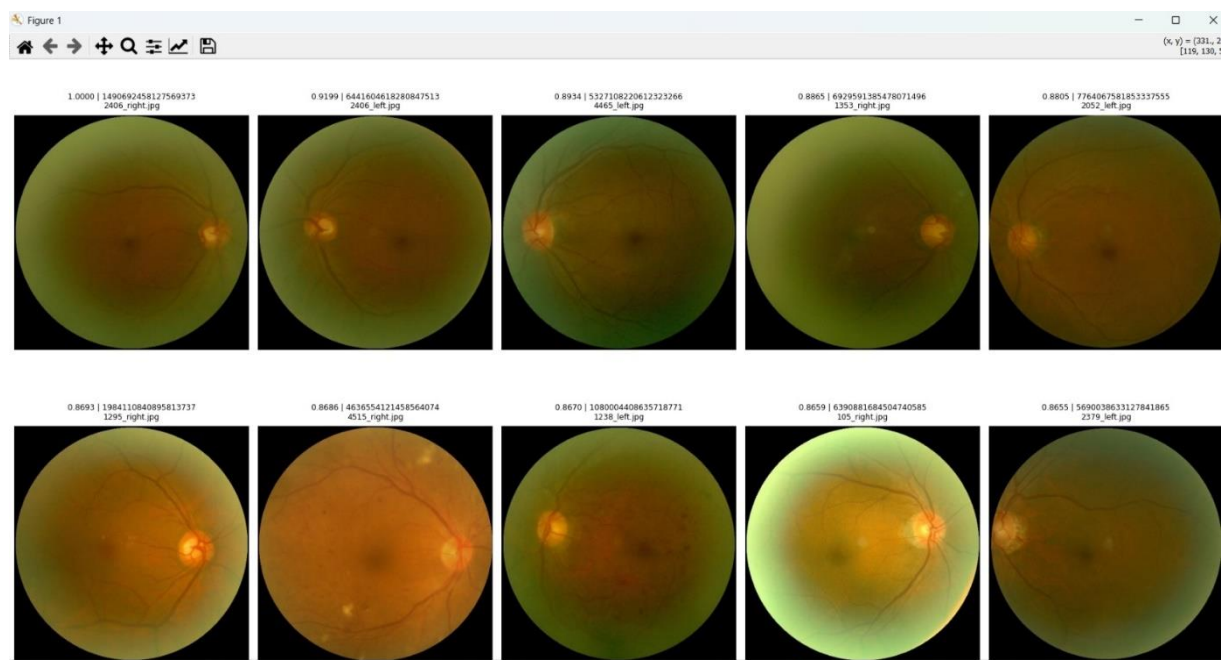
Für die exakte Suche im Arbeitsspeicher laden wir alle vorhandenen .npy-Dateien seitenweise ein, normalisieren die Vektoren und stapeln sie zu einer großen Matrix. Das passiert in `_load_all_embeds_from_npy`, und wir bauen daraus einen einfachen RAM-Index über `EmbedIndexExact`. Die Suche reduziert sich so auf ein Matrix-Vektor-Produkt und eine Top-K-Selektion, was in der Methode `search` der Klasse gelöst ist. Wenn wir stattdessen eine schnelle, approximative Suche über den gesamten Korpus brauchen, bauen wir einen HNSW-Index. Wir streamen dafür die Embeddings von der

Platte, normalisieren sie, fügen sie dem Graphen hinzu und speichern Index und ID-Mapping auf die Festplatte. Der Aufbau steckt in `build_hnsw_index`, das Laden und die Suchparameter steuern wir in `load_hnsw_index`, und die Abfrage mit Rückführung auf Dateipfade übernehmen wir in `search_hnsw`.

Auch hier steuern wir alles über das Kommandozeilen-Interface. Wir haben `--stage embed`, womit wir fehlende Embeddings erkennen, berechnen und als .npy ablegen, `--stage build_index_hnsw`, womit wir aus den abgelegten Vektoren den Graphindex konstruieren, und `--stage search_hnsw` sowie `--stage search_exact`, womit wir wahlweise über den HNSW-Index oder die RAM-Matrix suchen. Die Parameter `--outdir`, `--shard_mod`, `--shard_width` und `--dtype` legen fest, wo die Dateien landen, wie wir sharden und ob wir Speicher mit fp16 sparen oder auf fp32 setzen.

Ergebniss:

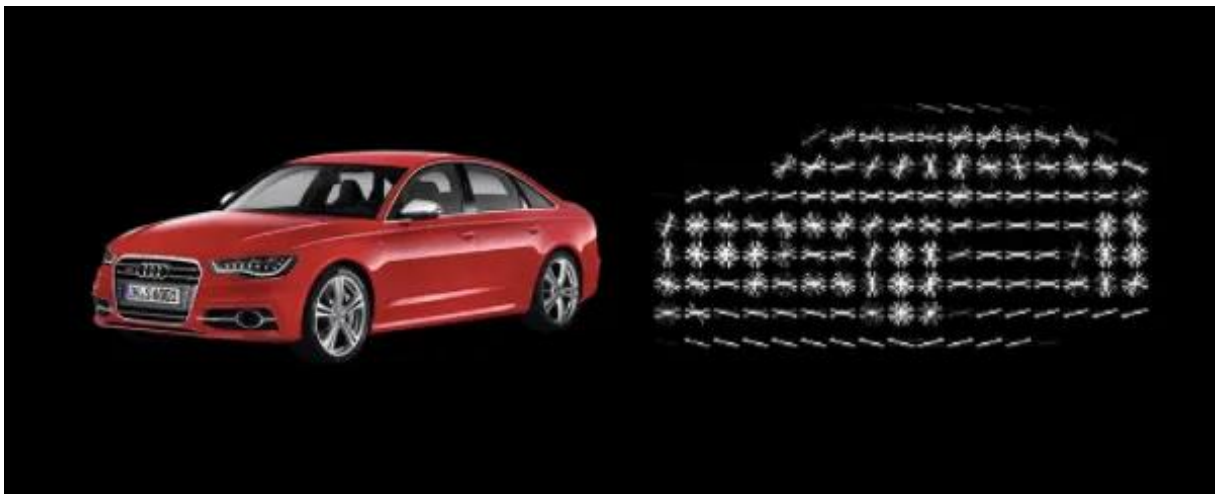
Das Bild „...3732406_right.jpg“ oben links ist das Ausgangsbild.



<https://medium.com/image-processing-with-python/efficientnetb0-architecture-stem-layer-496c7911a62d>

3.3 Histogram of Oriented Gradients

Neben den Deep-Learning-Embeddings haben wir auch eine klassische Methode umgesetzt, um Bildähnlichkeiten zu erfassen: das Histogramm of Oriented Gradients (HOG). Dieses Verfahren extrahiert Kanteninformationen, indem es Gradientenrichtungen in lokalen Bildbereichen zählt und daraus einen Vektor erstellt. So entsteht ein kompakter Deskriptor, der besonders sensitiv für die Form und Struktur von Objekten ist.



In unserem Code haben wir dafür zunächst einen globalen HOG-Descriptor definiert, der mit festen Parametern wie Blockgröße, Zellen und Binanzahl konfiguriert wird. Diese Werte sorgen dafür, dass alle Bilder den gleichen Vektorumfang haben, unabhängig von ihrer ursprünglichen Größe. Vor der Berechnung wandeln wir jedes Bild in Graustufen um und skalieren es auf 64x64 Pixel. Dieser Schritt kann wahlweise auf der CPU oder, wenn verfügbar, teilweise auch über CUDA auf der GPU durchgeführt werden. Die Vorverarbeitung und eigentliche HOG-Berechnung sind in den Funktionen `_preprocess_gray_cpu`, `_preprocess_gray_gpu` und `_compute_hog_from_bgr` umgesetzt. Das Ergebnis wird auf die Länge eins normalisiert, damit wir später direkt die Kosinusähnlichkeit als Ähnlichkeitsmaß verwenden können.

Für die Indexierung der gesamten Bilddatenbank haben wir die Funktion `index_images_hog` implementiert. Sie durchsucht entweder die Datenbank nach Bildern ohne HOG-Vektor oder scannt das Dateisystem, wenn noch keine Einträge vorhanden sind. Die Berechnung der Deskriptoren erfolgt parallel über mehrere Prozesse, um die große Menge an Bildern effizient zu verarbeiten. Nach der Extraktion werden die HOG-Vektoren im Feld `hog_vector` der Datenbank gespeichert. Dieser Ablauf erlaubt es uns, große Bildmengen wie die geforderten 500 GB systematisch und ohne manuelles Eingreifen vorzubereiten.

Die Ähnlichkeitsmessung selbst erfolgt über das Skalarprodukt der normierten Vektoren. Dafür haben wir die Funktion `hog_similarity` und darauf aufbauend die Methode `find_similar_hog_linear` entwickelt. Hierbei wird ein Query-Bild in einen HOG-Vektor umgewandelt und mit allen gespeicherten Vektoren verglichen. Das Verfahren liefert exakte Treffer, ist aber bei sehr großen Datenmengen langsamer, da jeder Vergleich einzeln durchgeführt wird.

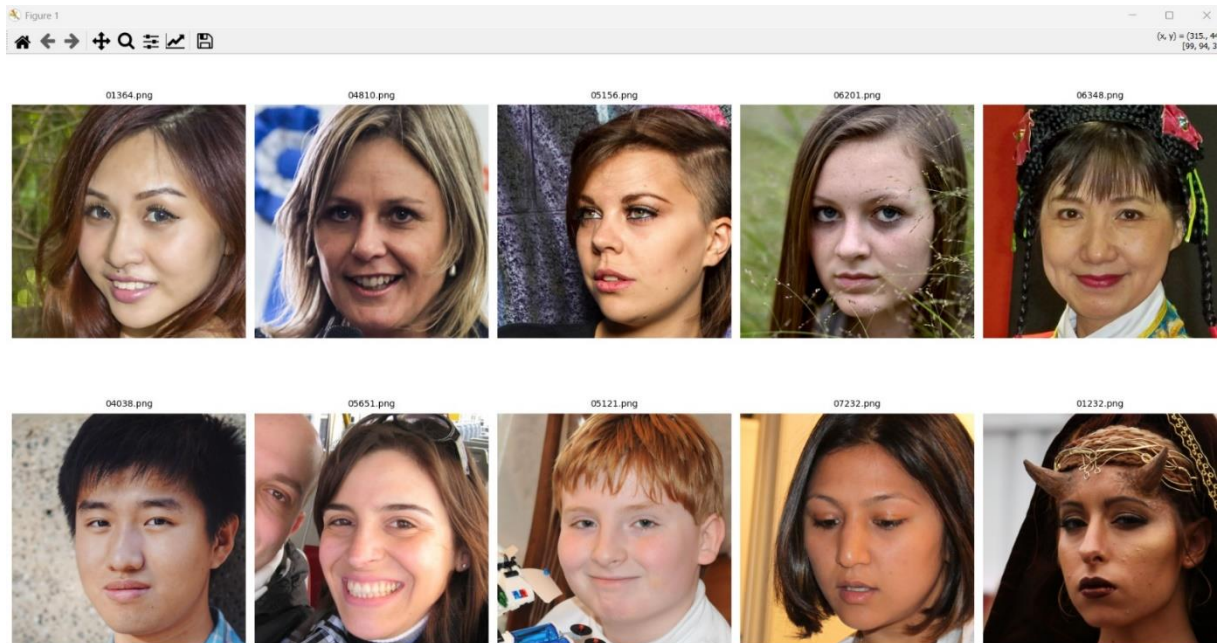
Um auch bei Millionen von Bildern schnelle Anfragen zu ermöglichen, haben wir zusätzlich eine Integration mit FAISS eingebaut. Mit der Funktion `build_faiss_from_db` können wir die HOG-Vektoren aus der Datenbank in einen effizienten Index überführen. Dieser Index unterstützt sowohl eine flache Suche als auch ein hierarchisches IVF-Verfahren, das die Vektoren in Clustern organisiert und dadurch deutlich beschleunigte Suchzeiten bietet. Für die Suche wird dann `search_faiss` verwendet, das die Top-K ähnlichsten Bilder in wenigen Millisekunden zurückgibt.

Wie schon bei den Deep-Learning-Embeddings steuern wir auch hier alle Schritte über das Command Line Interface. Mit Parametern wie `--stage index` werden HOG-Vektoren berechnet und abgespeichert, `--stage search_linear` startet eine exakte Suche, während `--stage build_faiss` und `--stage search_faiss` die schnelle Annäherungssuche mit FAISS nutzen. So können wir flexibel je nach Datenmenge zwischen exakter und beschleunigter Suche wählen.

<https://medium.com/analytics-vidhya/a-gentle-introduction-into-the-histogram-of-oriented-gradients-fdee9ed8f2aa>

Ergebniss:

Das Bild „01364.png“ oben links ist das Ausgangsbild.



3.3 Ergebnisbewertung der Suchmethoden

Für die Bewertung der drei eingesetzten Verfahren sowie der kombinierten Suche wurde eine Beispielabfrage durchgeführt, bei der ein Query-Bild gegen den Datensatz getestet und die fünf ähnlichsten Treffer ermittelt wurden. Die Suche konnte wahlweise mit einzelnen Metriken oder mit einer gewichteten Kombination erfolgen.

Die Resultate entsprachen in vielen Fällen den Erwartungen. Mit den Deep-Learning-Embeddings ließen sich meist sehr passende Treffer erzielen, die nicht nur farbliche, sondern auch inhaltliche Ähnlichkeiten widerspiegeln. So wurden beispielsweise Bilder von Menschen im Regen oder nassen Straßen zuverlässig als ähnlich erkannt. Dieses Verhalten bestätigt die Annahme, dass Embeddings semantische Eigenschaften von Bildern erfassen und nicht auf rein visuelle Merkmale wie Farbe oder Textur beschränkt sind.

Teilweise traten jedoch auch unerwartete Ergebnisse auf. Die Suche mit HSV-Vektoren bevorzugt stark ähnliche Farbstimmungen, sodass Bilder mit gleichem Farbton oder Look erscheinen, auch wenn die Motive inhaltlich unterschiedlich sind. HOG reagiert dagegen vor allem auf Kanten und Strukturen. Bei Szenen mit Wasser oder Regentropfen führte dies zu Treffern, die zwar ähnliche Texturen, jedoch nicht den gleichen Bildinhalt aufwiesen. Einzelne Ausreißer in den Ergebnissen lassen sich somit durch Farbbias bei HSV, Texturbias bei HOG oder durch Artefakte und ungewöhnliche Posen bei den Embeddings erklären.

Die Stärken der Verfahren zeigen sich in spezifischen Situationen. HSV liefert sinnvolle Treffer, wenn das Query durch dominante Farben geprägt ist, etwa bei einem roten Regenmantel. HOG erzielt gute Ergebnisse, wenn klare Konturen wie Fahrzeuge oder Personen im Vordergrund stehen. Embeddings überzeugen insgesamt am meisten, da sie auch semantisch ähnliche Inhalte erkennen, selbst wenn sich Farbgestaltung oder Textur stark unterscheiden.

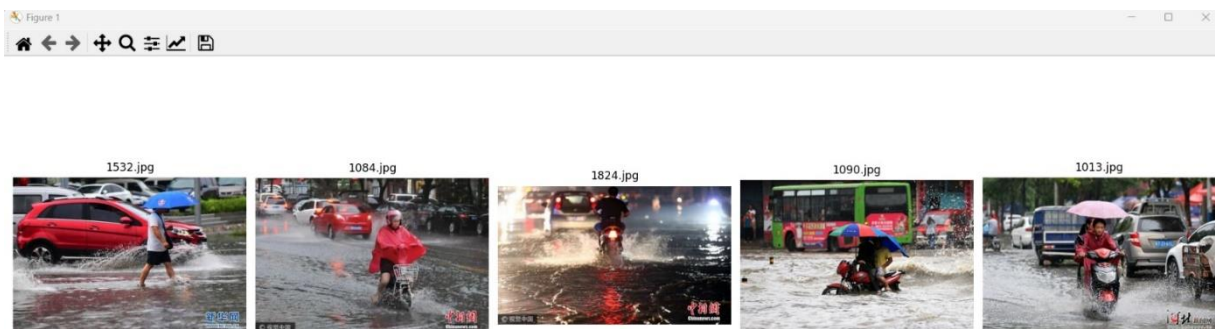
Die kombinierte Suche hat in den Experimenten zu den besten Gesamtergebnissen geführt. Hierbei werden die Scores von Embeddings, HSV und HOG normalisiert und mit gleichen Gewichten gemischt. Dadurch werden Schwächen einzelner Verfahren abgefedert und die Trefferlisten robuster. In vielen Fällen steigt die Qualität der Top-5 Ergebnisse, da sowohl Farbe, Struktur als auch Inhalt berücksichtigt werden. Die Gewichtung lässt sich zudem flexibel anpassen, etwa mit höherem HSV-Anteil bei Mode- oder Designanwendungen oder stärkerem Fokus auf Embeddings und HOG bei der Objekterkennung.

Zusammenfassend lässt sich festhalten, dass die Embeddings die besten Einzelresultate liefern, da sie semantische Ähnlichkeiten zuverlässig erfassen und durch Indexierung in

Millisekunden abgefragt werden können. Die Kombination der Verfahren verbessert die Robustheit, da sie Farb- und Strukturinformationen ergänzt und dadurch in Sonderfällen stabilere Ergebnisse liefert. Verwunderliche Einzeltreffer lassen sich meist nachvollziehen und treten durch die Score-Fusion deutlich seltener auf.

3.4 Zusammenführung

Das Ergebniss von HSV HOG und Deep-Learn-Embeddings zusammen:



4 Leistungsanalyse und Laufzeitorientierung

4.1 Engpässe und Messergebnisse

Die Laufzeitanalyse zeigt deutliche Unterschiede zwischen den verschiedenen Verfahren. Bei der Berechnung der Deep-Learning-Embeddings mit EfficientNet-B0 auf der GPU konnten wir im Mittel rund 31,6 Bilder pro Sekunde verarbeiten, wenn wir mit einer Batchgröße von 512 und reduzierter Genauigkeit im FP16-Modus gearbeitet haben. So wurden beispielsweise 8 706 von insgesamt 50 000 Bildern in etwa vier Minuten und zwanzig Sekunden verarbeitet. Der Hauptengpass liegt hier klar im GPU-Forward-Pass, gefolgt von der Bilddekodierung und dem Ein- und Ausgabesystem, während das Speichern der .npy-Dateien nur nachgelagert ins Gewicht fällt.

Die farbbasierte HSV-Berechnung mit OpenCV auf der CPU erwies sich als besonders schnell. Für zwanzigtausend Bilder betrug die gesamte Rechenzeit ungefähr zwei Minuten, was einem Durchsatz von 166 Bildern pro Sekunde entspricht, mit kurzzeitigen Spitzenwerten bis knapp unter eintausend Bilder pro Sekunde. Hier zeigt sich, dass der Prozess primär durch das Dateieinlesen und nicht durch die eigentliche Berechnung begrenzt ist.

Die HOG-Deskriptoren wurden ebenfalls auf der CPU mit OpenCV berechnet. Für zwanzigtausend Bilder benötigte das System eine Minute und siebenunddreißig Sekunden, was einer Geschwindigkeit von etwa 204 Bildern pro Sekunde entspricht. Zwischenzeitlich konnten sogar bis zu 380 Bilder pro Sekunde erreicht werden. Da HOG rechenintensiver als HSV ist, liegt der Engpass weniger im I/O als in der eigentlichen Gradientenberechnung.

Für den Aufbau des HNSW-Index zeigten sich andere Engpässe. Das Laden der Embeddings erfolgte mit etwa 58 Bildern pro Sekunde, der eigentliche Aufbau des Graphen war jedoch der zeitintensivste Schritt. Damit verschiebt sich hier der Flaschenhals klar in die Indexkonstruktion.

Die größten Zeitanteile entfallen damit beim Deep-Learning-Ansatz auf den GPU-Forward-Pass und die Bilddekodierung, beim Indexaufbau auf die Graphkonstruktion.

4.2 Optimierungen

Um diese Engpässe abzumildern, haben wir mehrere Maßnahmen umgesetzt. Durch eine strikte Kontrolle der Threadanzahl in OMP, MKL, OpenBLAS und OpenCV konnten wir verhindern, dass sich Multiprocessing und interne Threadpools gegenseitig blockieren. Für das Einlesen von Bildern setzen wir auf OpenCV anstelle von Pillow und nutzen bis zu 24 parallele Worker, was die Pipeline deutlich beschleunigt. Auf der GPU erreichen wir durch Mixed Precision mit FP16 und große Batches einen höheren Durchsatz.

Auch die Datenbankzugriffe wurden optimiert. Durch gebündelte Commits und den Einsatz von WAL-Modus und passenden PRAGMAs konnten wir Schreiboperationen beschleunigen und Locking reduzieren. Um Speicherengpässe zu vermeiden, werden Embeddings nicht in einer großen Datei abgelegt, sondern in geshardeten .npy-Dateien gespeichert. Für die Robustheit und Kontrolle der Pipeline haben wir die Verarbeitung in Stages unterteilt, die mit Limit-Parametern und Fortschrittsanzeigen flexibel gesteuert werden können.

Darüber hinaus haben wir die Ähnlichkeitsberechnung bei HOG stabilisiert, indem wir die Normierung der Vektoren abgesichert haben, sodass numerische Probleme vermieden werden. Weitere Optimierungen umfassen den Einsatz von vektorisierten NumPy-Operationen, die Verwendung von Pinned Memory für schnelleren GPU-Transfer sowie die Beschleunigung der Nachbarschaftssuche über HNSW.

5. Feasibility analysis/Discussion

Die durchgeführten Experimente zeigen, dass alle eingesetzten Verfahren grundsätzlich schnell genug sind, um in einer Bildsuchmaschine eingesetzt zu werden. Die auf Farben und Texturen basierenden Methoden HSV und HOG erreichen auf der CPU Geschwindigkeiten zwischen einhundert und fast eintausend Bildern pro Sekunde. Damit eignen sie sich sehr gut für einfache Farbsuchen oder die Erkennung von Texturähnlichkeiten. Die Deep-Learning-Embeddings sind zwar in der Erstellung deutlich langsamer, ermöglichen jedoch eine semantische Suche mit wesentlich höherer Aussagekraft. Nach dem Aufbau eines HNSW-Index liegen die Antwortzeiten für einzelne Suchanfragen im Bereich von einhundert bis dreihundert Millisekunden, was für eine interaktive Nutzung problemlos ausreichend ist.

Für einen Datensatz von siebenhunderfünfzigtausend Bildern ergibt sich bei einer Verarbeitungsgeschwindigkeit von rund 31,6 Bildern pro Sekunde eine reine Berechnungszeit von etwa sechs bis acht Stunden für die Embeddings auf einer einzelnen GPU. Hinzu kommt der zeitaufwändige Aufbau des Index, dessen Dauer je nach Parametern variieren kann. Der Speicherbedarf bleibt in einem handhabbaren Rahmen: im FP16-Format benötigt jedes Embedding etwa 2,5 Kilobyte, was bei siebenhunderfünfzigtausend Bildern einer Gesamtdatenmenge von ungefähr 1,9 Gigabyte entspricht. Der dazugehörige HNSW-Index belegt weitere 3,8 Gigabyte für die Vektoren sowie etwa 100 Megabyte für den Graphen, sodass insgesamt rund vier bis fünf Gigabyte RAM erforderlich sind. Der Platzbedarf der eigentlichen Bilddateien hängt stark von deren Größe und vom eingesetzten Speichersystem ab, ist jedoch für die Verarbeitungsgeschwindigkeit ebenfalls von Bedeutung, da der Ein- und Ausgabedurchsatz entscheidend ist.

Die Skalierbarkeit ist bis zu einer Größenordnung von etwa einer Million Bildern auf einer leistungsfähigen Workstation mit ausreichend Hauptspeicher und schneller SSD oder NVMe problemlos gegeben. Jenseits dieser Grenze, insbesondere im Bereich von ein bis fünf Millionen Bildern, geraten sowohl der Speicherverbrauch als auch die Dauer des Indexaufbaus an kritische Punkte. In diesem Fall wären verteilte Verfahren oder weiter optimierte Methoden wie FAISS mit IVF oder PQ sowie Sharding notwendig, um die Datenmenge effizient zu handhaben.

Dennoch gibt es Einschränkungen. Die Erstellung der Embeddings ist stark GPU-gebunden und erfordert mehrere Stunden bei großen Datenbeständen. Auch der Aufbau des HNSW-Index ist sowohl speicher- als auch rechenintensiv. SQLite genügt für das lokale Handling der Metadaten, ist jedoch nicht für Szenarien mit vielen parallelen Nutzern geeignet. Darüber hinaus ist eine schnelle SSD oder NVMe zwingend erforderlich, da herkömmliche Festplatten für die notwendige I/O-Geschwindigkeit nicht ausreichen. Schließlich muss der Parameter `efSearch` im Index so gewählt werden, dass ein guter Kompromiss zwischen Recall und Latenz erreicht wird.

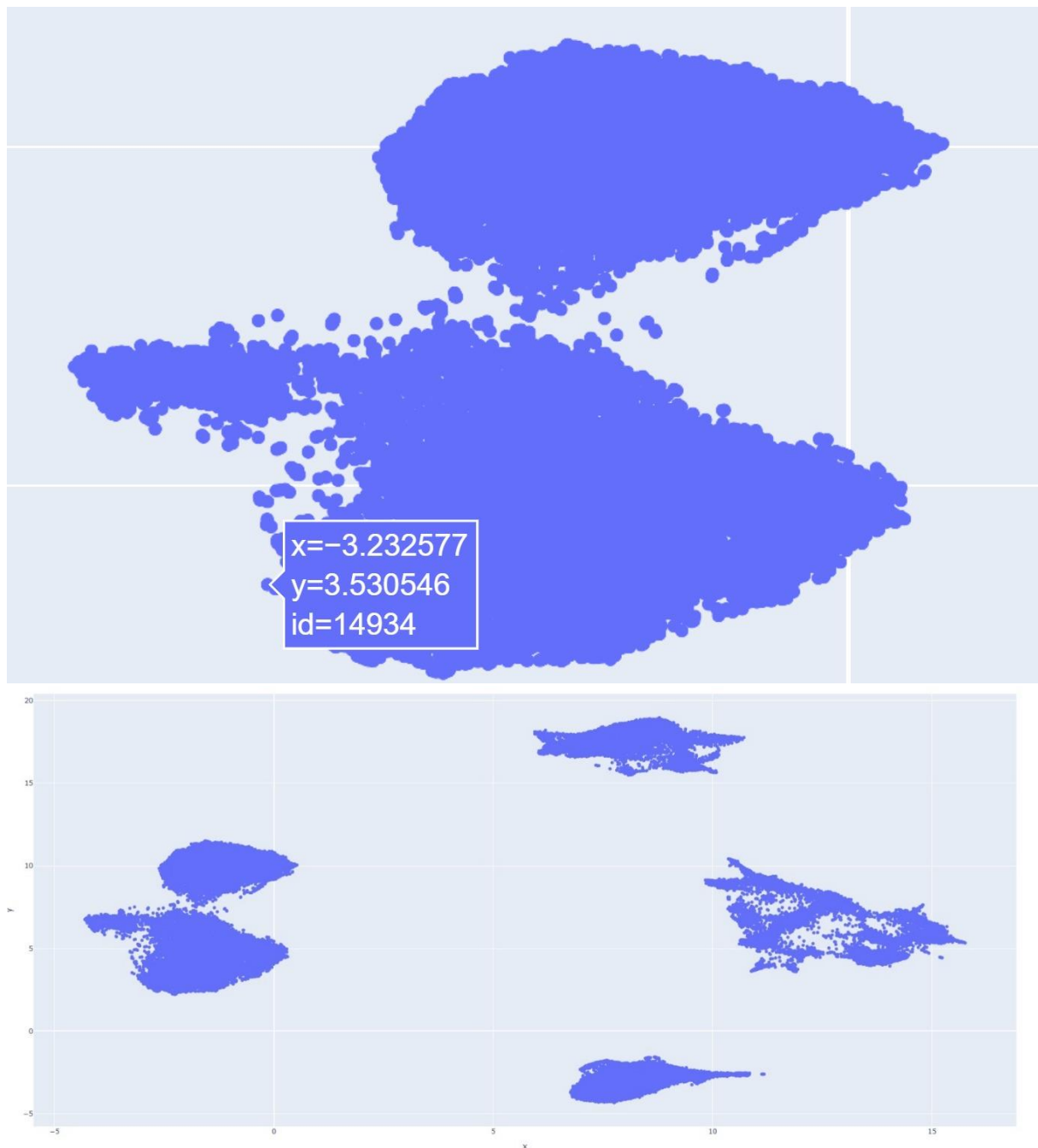
Insgesamt zeigt sich, dass das System mit den beschriebenen Optimierungen in der Lage ist, Datensätze von der Größenordnung von siebenhunderfünfzigtausend Bildern effizient zu verwalten. Die Abfragen laufen im sub-sekündigen Bereich ab, sodass eine interaktive Suche möglich ist. Die größten Limitierungen liegen in der erforderlichen GPU-Zeit für die Berechnung der Embeddings und im hohen Speicherbedarf während des Indexaufbaus. Trotz dieser Grenzen ist das System damit klar für den Einsatz in realistischen Big-Data-Szenarien geeignet.

6. Big Data Image Analysis

6.1 Ergebnisse

Die Projektion mit UMAP zeigt deutlich, dass die Embeddings in mehrere klar abgegrenzte Cluster zerfallen. Innerhalb dieser Cluster liegen die Punkte dicht beieinander, während zwischen den Gruppen größere Abstände bestehen. In der Übersichtsgrafik wird sichtbar, dass sich semantisch ähnliche Bilder zuverlässig zusammenlagern und damit eine erkennbare Struktur im Datensatz entsteht. Eine Detailansicht des ersten Clusters zeigt, dass hier Bilder mit ähnlichen Inhalten gruppiert wurden, beispielsweise gleiche Objekte, vergleichbare Szenen oder ähnliche Farbverteilungen. Eine weitere Detailansicht eines zweiten Clusters bestätigt diese Beobachtung und macht deutlich, dass die lokale Nachbarschaftsstruktur sehr stabil abgebildet wird. Auffällig sind einzelne Ausreißer, die abseits der Hauptcluster liegen. Diese Fälle könnten auf ungewöhnliche Bildinhalte oder verrauschte Embeddings hinweisen.





6.1 Diskussion

Die Analyse zeigt, dass die Kombination aus PCA und UMAP eine sinnvolle Methode ist, um hochdimensionale Bilddaten zu visualisieren. Ähnliche Bilder liegen eng beieinander, während unähnliche Bilder weiter auseinanderfallen. Die Cluster sind klar erkennbar und visuell interpretierbar, was bestätigt, dass die Embeddings tatsächlich semantische Ähnlichkeiten erfassen. Besonders positiv ist, dass die PCA-Reduktion die Rechenzeit stark verkürzt hat, ohne die inhaltliche Struktur wesentlich zu verändern. UMAP hat eine klare Clusterstruktur erzeugt, die auch aus inhaltlicher Sicht plausibel wirkt. Gleichzeitig muss berücksichtigt werden, dass die globale Anordnung der Cluster nicht zu stark interpretiert werden sollte, da UMAP primär lokale Nachbarschaften optimiert. Außerdem reagieren die Ergebnisse sensibel auf Parameter wie die Zahl der Nachbarn und den Minimalabstand, was bei anderen Konfigurationen zu leicht abweichenden Layouts führt.